

Task 1: To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```
GNU nano 7.2 cd_test.c *
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

int main() {
    char current[PATH_MAX];
    char previous[PATH_MAX];

    getcwd(current, sizeof(current));
    strcpy(previous, current);

    printf("Current directory: %s\n", current);

    char path[PATH_MAX];

    printf("Enter directory: ");
    scanf("%s", path);

    if (chdir(path) == 0) {
        strcpy(previous, current);
        getcwd(current, sizeof(current));

        printf("Directory changed successfully\n");
        printf("Previous directory: %s\n", previous);
    }
}
```

```
printf("Current directory: %s\n", current);

char path[PATH_MAX];

printf("Enter directory: ");
scanf("%s", path);

if (chdir(path) == 0) {
    strcpy(previous, current);
    getcwd(current, sizeof(current));

    printf("Directory changed successfully\n");
    printf("Previous directory: %s\n", previous);
    printf("Current directory: %s\n", current);
} else {
    perror("Directory change failed");
}

return 0;
}
```

```

shreyas@Grigori:~/skill_session_8$ mkdir -p ~/skill_session_9
shreyas@Grigori:~/skill_session_8$ cd ..
shreyas@Grigori:~$ mkdir -p ~/skill_session_9
shreyas@Grigori:~$ cd ~/skill_session_9
shreyas@Grigori:~/skill_session_9$ nano cd_test.c
shreyas@Grigori:~/skill_session_9$ gcc cd_test.c -o cd_test
shreyas@Grigori:~/skill_session_9$ ./cd_test
Current directory: /home/shreyas/skill_session_9
Enter directory: skill_session_8
Directory change failed: No such file or directory
shreyas@Grigori:~/skill_session_9$ DSEDBD
DSEDBD: command not found
shreyas@Grigori:~/skill_session_9$ ./cd_test
Current directory: /home/shreyas/skill_session_9
Enter directory: /tmp
Directory changed successfully
Previous directory: /home/shreyas/skill_session_9
Current directory: /tmp
shreyas@Grigori:~/skill_session_9$ /abcxyz
-bash: /abcxyz: No such file or directory
shreyas@Grigori:~/skill_session_9$ pwd
/home/shreyas/skill_session_9

```

Task 2: To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>

void cmd_pwd() {
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
}

void cmd_cd(char *path) {
    if (chdir(path) != 0)
        perror("cd");
}

void cmd_echo(char *text) {
    printf("%s\n", text);
}

typedef void (*builtin_function)(char *);

typedef struct {
    char *name;
    builtin_function function;
} Builtin;

```

```

        if (!found)
            printf("Invalid command: %s\n", command);
    }

    return 0;
}

```

```

shreyas@Grigori:~/skill_session_9$ nano builtins.c
shreyas@Grigori:~/skill_session_9$ gcc builtins.c -o builtins
shreyas@Grigori:~/skill_session_9$ ./builtins
shell> pwd
/home/shreyas/skill_session_9
shell> ls
^C
shreyas@Grigori:~/skill_session_9$ cd /tmp
shreyas@Grigori:/tmp$ cd -
/home/shreyas/skill_session_9
shreyas@Grigori:~/skill_session_9$ |

```

```

GNU nano 7.2                                builtins.c
char command[100];
char argument[200];

while (1) {

    printf("shell> ");
    scanf("%99s", command);

    if (strcmp(command, "exit") == 0)
        break;

    if (strcmp(command, "pwd") == 0) {
        strcpy(argument, "");
    } else {
        scanf("%199s", argument);
    }

    int found = 0;

    for (int i = 0; i < 3; i++) {

        if (strcmp(command, builtins[i].name) == 0) {
            builtins[i].function(argument);
            found = 1;
            break;
        }
    }
}

```